

JFEM Workspace Structure, Readable Cards, Runnable SOLs, and Code Complexity

Software-engineering inventory of the current repository and development workspace

JFEM development documentation

Generated 2026-06-01

Purpose of This Deck

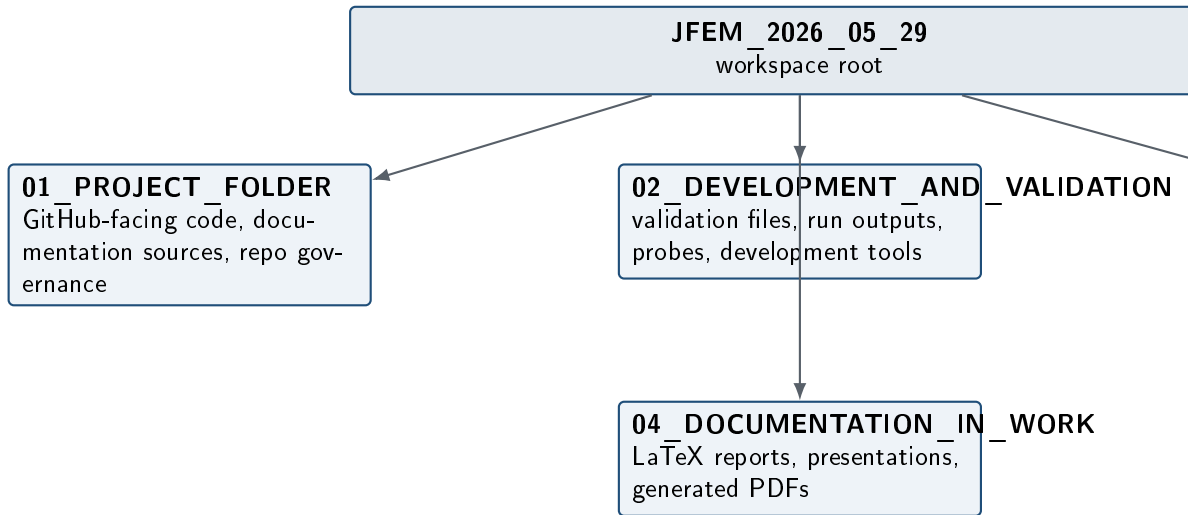
What is documented

- Workspace shape and repository boundaries.
- Distributable JFEM package anatomy.
- Julia source size by layer and by largest module.
- Readable Nastran-style cards.
- Runnable solution sequences and execution modes.

Why it matters

- Helps communicate the codebase complexity.
- Helps future agents avoid mixing repo code with scratch data.
- Identifies engineering hotspots before new SOLs and formulations.
- Provides a frozen technical baseline for documentation.

Top-Level Workspace Structure



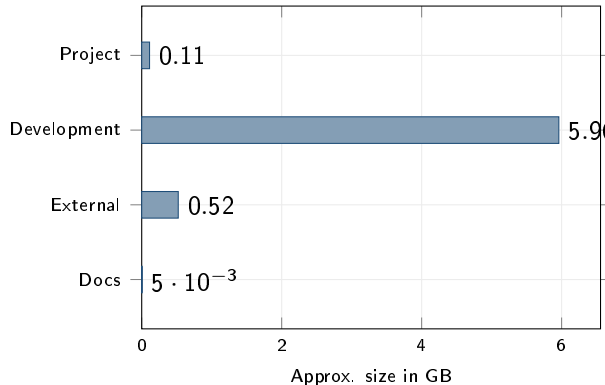
The key principle is separation: public repository, validation laboratory, external tools, and generated documentation live in

Workspace Size Snapshot

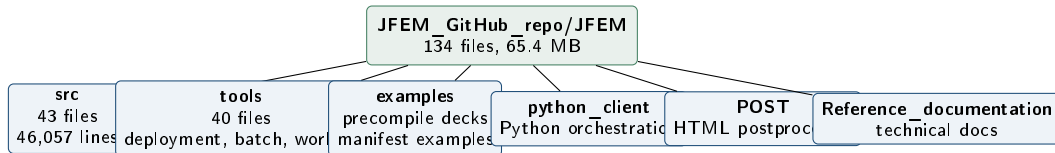
Folder	Files	Approx. size
.claude	8	32 KB
01_PROJECT_FOLDER	332	112 MB
02_DEVELOPMENT_AND_VALIDATION	24,619	5.96 GB
03_EXTERNAL_TOOLS	18,599	522 MB
04_DOCUMENTATION_IN_WORK	62	5 MB

Reading the numbers

Most disk volume is intentionally outside the public repository: validation files, run outputs, solver binaries, and external references.



Distributable Repository Anatomy



Repository role

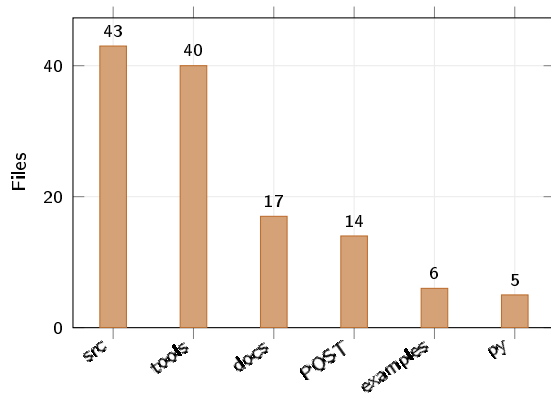
This is the folder intended for GitHub publication and user installation.

Non-repository role

Large validation datasets and external solvers remain outside this tree.

Repository File Distribution

Top-level folder	Files	Size
src	43	1.9 MB
tools	40	10.5 MB
Reference_documentation	17	46.1 MB
POST	14	6.8 MB
examples	6	3 KB
python_client	5	28 KB



OpenJFEM Package Identity

Julia package

- Package name: `OpenJFEM`.
- Entry points: package API and `src/main.jl`.
- Project file declares solver, export, AD, HDF5, and worker dependencies.

43

Julia source files

Project dependencies

AlgebraicMultigrid, ForwardDiff, HDF5, HTTP, IterativeSolvers, JSON, KrylovKit, MsgPack, NPZ, PrecompileTools, StaticArrays, WriteVTK, LinearAlgebra, SparseArrays, Statistics

46,057

total source lines

1,260

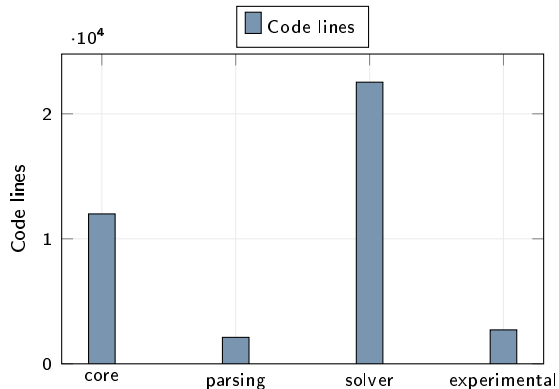
function definitions

Julia Source Size by Layer

Layer	Files	Lines	Code	Funcs
Core/export/API	10	14,207	11,992	324
Parsing	11	2,427	2,116	87
Solver	17	26,119	22,530	787
Experimental	5	3,304	2,714	62
Total	43	46,057	39,352	1,260

Observation

The solver layer contains more than half the source and almost two thirds of the functions.

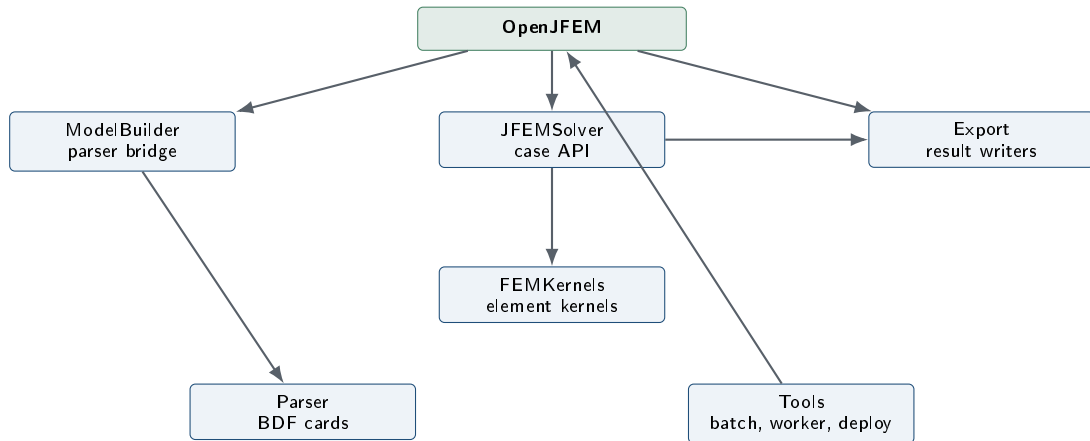


Largest Source Files

File	Lines	Code	Funcs	Layer	Main role
src/solver/assembly.jl	7,402	6,591	244	solver	Global matrix assembly and element dispatch.
src/FEMKernels.jl	6,353	5,238	119	core	Element stiffness, mass, shell and solid kernels.
src/solver/solve_case.jl	4,063	3,560	40	solver	SOL dispatch, subcase orchestration, result export control.
src/Export.jl	3,294	2,933	110	core	JSON, VTK, HDF5, binary and report export surface.
src/solver/optimize_thickness.jl	3,156	2,792	78	solver	Thickness optimization and SOL200-related workflows.
src/solver/adjoint_buckling.jl	2,717	2,347	65	solver	Buckling adjoints and sensitivities.
src/JFEMSolver.jl	2,188	1,907	43	core	Public solver API and package-facing orchestration.

These files are the main engineering hotspots for future modularization and regression-guarding.

Include Graph: Package Level



Include Graph: Parser and Solver Detail

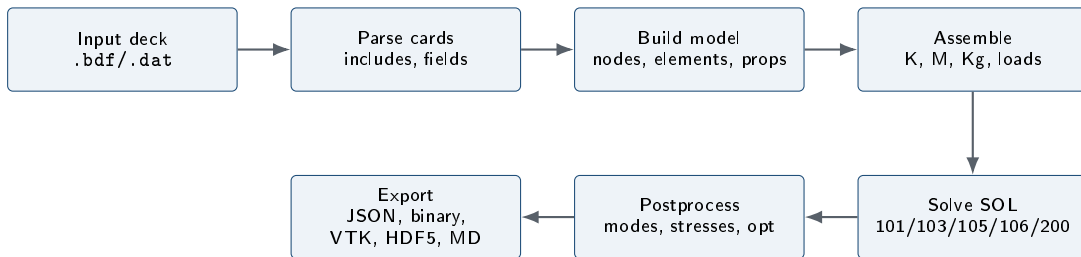
Parser modules

- `parse_bdf.jl`: bulk entry driver.
- `cards_structs.jl`: typed card records.
- `bulk_parser_*.jl`: field parsing and large-field support.
- `card_discovery.jl`: card inventory and readable coverage.
- `include_resolver.jl`: include-file resolution.

Solver modules

- `assembly.jl`: matrices, loads, constraints.
- `solve_case.jl`: SOL 101/103/105/106/200 routing.
- `boundary_conditions.jl`: SPC/MPC/RBE handling.
- `adjoint_buckling.jl`: sensitivities.
- `optimize_thickness.jl`: optimization loop.
- `hdf5_nastran_export.jl`: Nastran-style HDF5 surface.

Execution Pipeline



Shared architecture

Every runnable SOL depends on the same card parser, model builder, assembly layer, boundary-condition treatment, and export surface.

Readable Model Card Inventory

72

card families detected

SOL 101/103/105

production focus

SOL 106/200

experimental or partial

Card families present in code

CBAR, CBEAM, CBUSH, CELAS1, CELAS2, CHEXA, CMASS1, CMASS2, CONM1, CONM2, CONROD, CORD1R, CORD2C, CORD2R, CORD2S, CPENTA, CQUAD4, CQUAD8, CROD, CSHEAR, CTETRA, CTRIA3, CTRIA6, DCONSTR, DESVAR, DMIG, DOPTPRM, DRESP1, DVMREL1, DVPREL1, EIGRL, FORCE, GRAV, GRDSET, GRID, LOAD, MAT1, MAT2, MAT8, MATT1, MOMENT, MPC, MPCADD, PARAM, PBAR, PBARL, PBEAM, PBEAML, PBUSH, PCOMP, PELAS, PLOAD, PLOAD1, PLOAD2, PLOAD4, PMASS, PROD, PSHEAR, PSHELL, PSOLID, RBAR, RBE1, RBE2, RBE3, RFORCE, RSPLINE, SPC, SPC1, SPCADD, TABLEM1, TEMP, TEMPD

CARDS: Geometry and Elements

Coordinates and grids

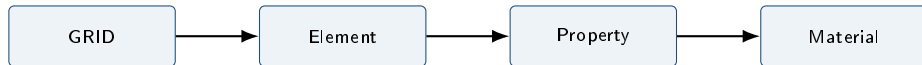
GRID
GRDSET
CORD1R
CORD2R
CORD2C
CORD2S

1D and lumped

CROD, CONROD
CBAR, CBEAM
CELAS1, CELAS2
CMASS1, CMASS2
CONM1, CONM2
CBUSH

Shells and solids

CTRIA3, CTRIA6
CQUAD4, CQUAD8
CSHEAR
CTETRA
CPENTA
CHEXA



CARDS: Properties, Materials, Loads

Properties

PSHELL
PCOMP
PSOLID
PROD
PBAR, PBARL
PBEAM, PBEAML
PELAS
PMASS
PBUSH
PSHEAR

Materials

MAT1
MAT2
MAT8
MATT1

Includes isotropic, orthotropic, and table-backed material data used by shell, solid, rod, beam, and composite workflows.

Loads and fields

FORCE
MOMENT
PLOAD
PLOAD1
PLOAD2
PLOAD4
GRAV
RFORCE
LOAD
TEMP, TEMPD
TABLEM1

CARDS: Constraints, Modal, Optimization

Constraints and couplings

SPC
SPC1
SPCADD
MPC
MPCADD
RBE1
RBE2
RBE3
RBAR
RSPLINE

Eigen and matrix

EIGRL
DMIG
PARAM

Supports modal and buckling
extraction controls plus imported
matrix data.

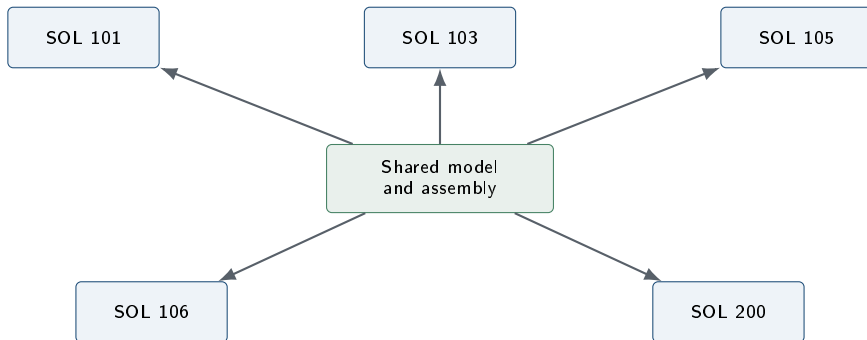
Optimization

DESVAR
DRESP1
DCONSTR
DVPREL1
DVMREL1
DOPTPRM

Used by the in-code thickness
optimization and SOL200-like
development path.

Runnable SOL Sequences

SOL	Purpose in JFEM	Maturity
SOL 101	Linear static analysis: displacement solution, loads, constraints, element recovery.	Active validation focus.
SOL 103	Normal modes: stiffness/mass assembly and eigenvalue extraction.	Supported.
SOL 105	Linear buckling: preload/static state, geometric stiffness, eigenproblem, mode export.	Primary parity target.
SOL 106	Nonlinear static development path.	Experimental.
SOL 200-lite	Optimization-oriented workflows, design variables, responses, adjoints.	Partial / evolving.



SOL Dispatch and Shared Infrastructure

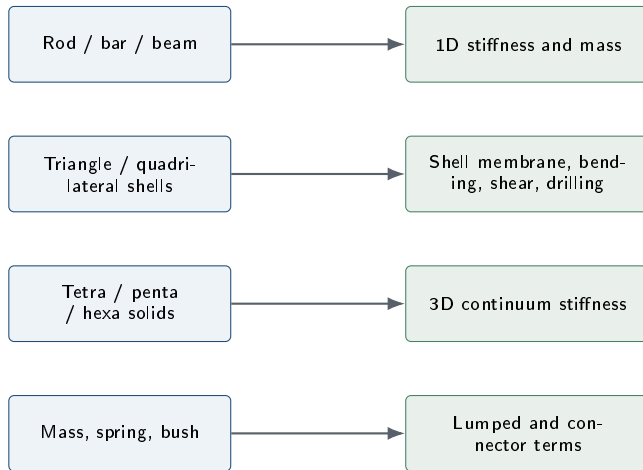
Common infrastructure

- Deck parsing and include resolution.
- Typed card storage.
- Element property/material lookup.
- Constraint elimination and MPC handling.
- Sparse matrix assembly.
- Result export and reporting.

SOL-specific work

- SOL 101: solve $Ku = f$.
- SOL 103: solve $K\phi = \lambda M\phi$.
- SOL 105: assemble K_g and solve $K\phi + \lambda K_g\phi = 0$.
- SOL 200: differentiate responses and update design variables.

Element/Formulation Coverage Map



Shell formulation work is the dominant parity lever for both SOL 101 displacement/stress recovery and SOL 105 geometric stiffness behavior.

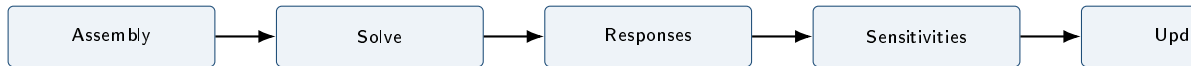
Core Solver Algorithms

Linear algebra

- Sparse global matrix assembly.
- Constraint reduction and recovery maps.
- Direct and iterative linear solves through Julia sparse tooling.
- Modal and buckling extraction through Krylov/eigen solvers.

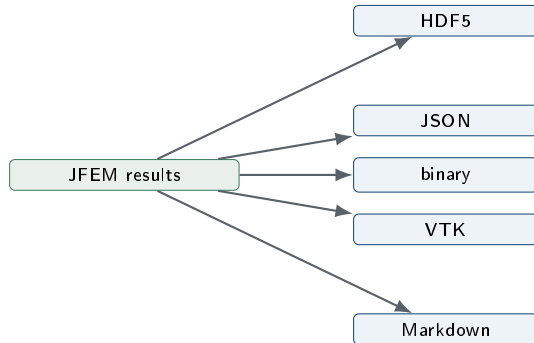
Sensitivity and optimization

- Forward-mode automatic differentiation support.
- Buckling adjoint sensitivities.
- Design variables and property/material relation cards.
- Thickness optimization workflow and reporting.



Output families

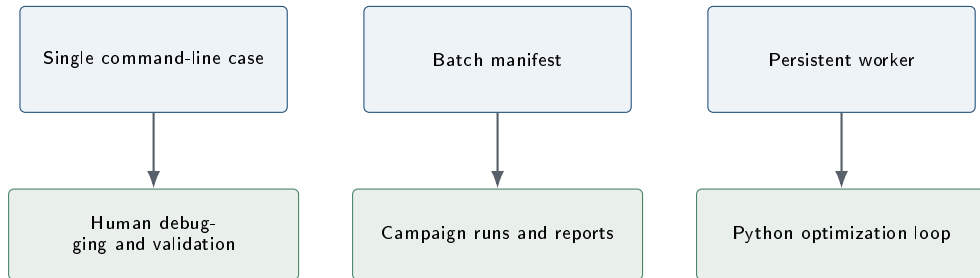
- JSON summary and model export.
- JFEM binary result files.
- VTK visualization output.
- HDF5/Nastran-style output workstream.
- Markdown reports.
- HTML postprocessor in POST.



Tooling and Execution Modes

Tool / mode	Purpose
tools/deploy_fast.jl	Broad precompile/deployment helper; can build fast startup workloads.
tools/precompile_sol105.jl	Representative precompile flow for buckling workloads.
tools/jfem_worker_jsonl.jl	Persistent worker for Python-driven optimization loops.
tools/run_batch_manifest.jl	Manifest-driven batch execution with explicit inputs and output folders.
tools/manifest_batch_core.jl	Shared batch manifest implementation.
tools/inspect_hdf5_schema.jl	HDF5 schema inspection and comparison utility.
python_client	Python-side client code for external orchestration.
examples/precompile	Bundled SOL 101/103/105 decks for warming representative code paths.

Execution Modes for Users and Automation



The fastest repeated workflow is the persistent worker, because Julia stays loaded and compiled between cases.

Software Engineering Hotspots

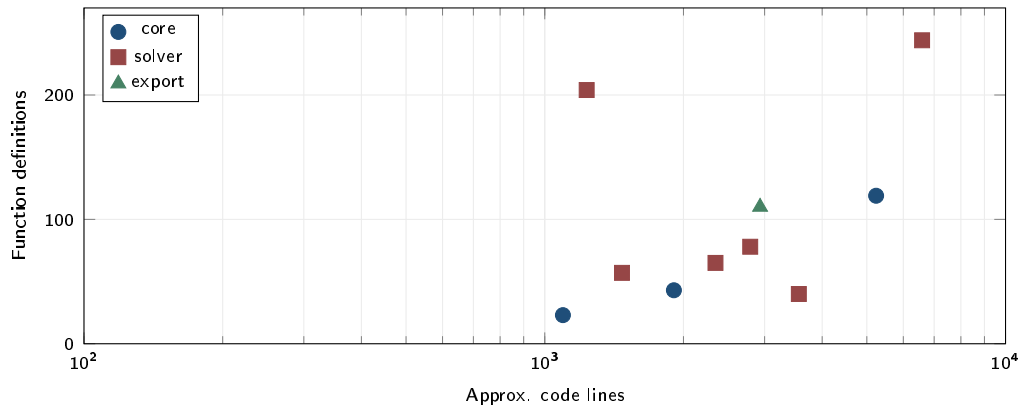
High-complexity modules

- `assembly.jl`: very large, many dispatch paths.
- `FEMKernels.jl`: formulation-critical and parity-sensitive.
- `solve_case.jl`: combines orchestration, SOL logic, export decisions.
- `Export.jl`: many formats and result variants.

Engineering risks

- Case-specific correction logic can hide formulation gaps.
- Large monolithic files make regressions harder to isolate.
- Export and solver paths should remain cleanly separated.
- Validation machinery must preserve mode and subcase identity.

Complexity Map



The outlier at high function count is solver helper infrastructure; the largest code body remains global assembly.

Maintainability Observations

Strengths

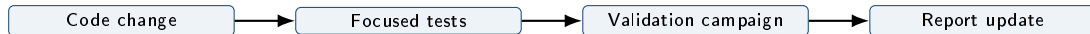
- Package form is in place.
- Repository has clear user tooling.
- Batch and worker execution support optimization use cases.
- Broad card inventory supports realistic imported decks.
- Validation workspace is separated from GitHub code.

Improvement opportunities

- Split large solver files by responsibility.
- Keep formulation options generic and geometry/material/load based.
- Add structured result objects for all SOLs.
- Enforce regression guards on SOL 101/103/105 together.
- Keep generated reports and run outputs outside the repository.

Recommended Governance for Future Work

- 1 Keep 01_PROJECT_FOLDER/JFEM_GitHub_repo/JFEM as the only GitHub upload surface.
- 2 Keep validation campaigns in 02_DEVELOPMENT_AND_VALIDATION.
- 3 Register every new validation case with purpose, source, expected result, and usefulness status.
- 4 Treat SOL 105 parity as a guard when modifying shell or static formulations.
- 5 Never tune by case name or dataset name; use only generic element, material, thickness, curvature, and load-state quantities.
- 6 Keep Nastran reference data and external solvers outside the public repository.
- 7 Preserve generated documentation sources and PDFs in 04_DOCUMENTATION_IN_WORK.



Summary

4

workspace zones

134

repo files

46k

Julia source lines

72

readable card families

5

SOL paths

1,260

function definitions

Engineering message

JFEM is now a package-shaped numerical solver with realistic Nastran-style input coverage, multiple execution modes, and a sizable solver kernel. The next growth phase should emphasize modularity, regression guards, and generic formulation improvements.